```
    node.left ← insert(node.left, newPoint, node.level +
1)          #5
    return node

else
     #6
    node.right ← insert(node.right, newPoint, node.level +
1)         #7
    return node
```

#1 Insert a point on the tree. The method will return a pointer to the root of the (sub)tree containing the new point. The level to which the node is added is set to `0` by default.

#2 If node is `null`, we are traversing an empty tree, so we have performed an unsuccessful search: we can therefore create a new node and return it, so the caller can store a reference to it. Notice how this works also when the tree is empty, creating and returning a new root.

#3 If the new point matches this node's point, we ran a successful search and we have a duplicate: here, we will just ignore duplicates, but you can handle it by changing these lines.

#4-#7 Otherwise, we need to compare the coordinates of target and node's point: we use the helper method previously defined, and check if it is lower than `0`, which would mean we have to take a left turn during tree traversal, and therefore we run the method recursively on the left branch, setting the result as the new left (or right) child of current node.